

The Aerial Guardian

Drone-Based Multi-Person Detection and Tracking

Siddhant Roy | Final Year ECE | VisDrone MOT Validation Set

1. Problem Statement

Drones create a really specific set of problems for computer vision. When a drone is at high altitude, persons in the frame can be as small as 8-10 pixels. Standard detectors like YOLO were trained on ground level images where persons are 200-400px tall, so they just don't work well at this scale out of the box. On top of that, the drone itself is moving, which means the background shifts between every frame. Most trackers assume the camera is static, so this movement breaks their predictions and causes ID switches. This project tries to fix all three of these problems while keeping the model small enough to actually run on a drone that too under 300 MB total.

2. Dataset

I used the VisDrone 2019 MOT Validation Set (Task 4). It has 7 video sequences from drone cameras covering different urban scenes — intersections, markets, open roads. The target classes are Pedestrian (class 1) and People (class 2). I treated both as the same Person category during inference since for tracking purposes they are the same thing.

Sequence	Person Detections	Role
uav0000086_00000_v	22,098	Train + Output Video
uav0000117_02622_v	9,670	Train
uav0000137_00458_v	9,299	Train
uav0000339_00001_v	5,483	Train
uav0000268_05773_v	1,984	Train
uav0000182_00000_v	1,175	Train
uav0000305_00000_v	603	Validation (see Section 4.1 — bad choice)

Table 1: Per-sequence person detection counts (from label analysis script)

2.5 Preprocessing Pipeline

VisDrone MOT annotations are not in YOLOv8 format, so I had to write a conversion script from scratch. The main difference is that VisDrone stores all frames of a video in one .txt file with pixel coordinates and 1-indexed classes, while YOLOv8 expects one .txt file per image with normalised centre coordinates and 0-indexed classes.

What preprocess.py does, step by step:

- Reads each annotation file and groups rows by frame_idx using a dictionary , so all rows belonging to frame 102 go together, frame 103 together, etc.
- Filters only person classes (row[7] == '1' or '2') and ignores everything else

- Converts top-left pixel coordinates to normalised centre coordinates: $x_center = (x + w/2) / img_width$, $y_center = (y + h/2) / img_height$
- Converts 1-indexed VisDrone classes to 0-indexed YOLO format: $cls = int(row[7]) - 1$
- Reads image dimensions dynamically for each sequence using PIL, I initially hardcoded 1344x756 but then fixed it
- Writes one .txt file per frame with zero-padded names like 0000001.txt to match image filenames exactly

Note: One thing that confused me initially, image coordinates have $y=0$ at the top and increase downward, which is opposite to normal maths. So $y_center = y + h/2$, not $y - h/2$.

3. Architecture

The pipeline runs in this order for each frame:

Step	Component	What it does
1	Frame read + grayscale	Prepare input for CMC optical flow
2	SAHI sliced inference	Run YOLOv8n on 6 tiles instead of full frame
3	Cross-tile NMS	Remove duplicate boxes at tile boundaries
4	CMC — Lucas-Kanade	Correct detection positions for camera movement
5	ByteTrack	Match detections to existing tracks, assign IDs
6	Draw output	Boxes + ID labels + trajectory tails on frame

Table 2: Pipeline steps per frame

3.1. Detection — YOLOv8n + SAHI

Why YOLOv8n: It is 6.2 MB. That is the main reason. The assignment said under 300 MB and it should be lightweight enough for a drone. YOLOv8n fits easily. It also has an anchor-free head and C2f backbone which gives it better accuracy than its size suggests. Since it is pretrained on COCO it already knows what a person looks like, I just needed to fine-tune it to recognise persons from drone altitude, not teach it from scratch.

Parameter	Value
Base model	yolov8n.pt (COCO pretrained)
Training data	6 VisDrone sequences (~2,600 frames)
Epochs / Batch / Image size	30 / 16 / 640x640
Optimizer	AdamW (auto-selected by ultralytics), LR=0.001667
Hardware	T4 GPU (Google Colab free tier)
Final model size	6.2 MB

Table 3: Fine-tuning details

Why SAHI: The core problem is that a person who is 10px tall in a 1920x1080 frame becomes roughly 3-4px after YOLO resizes the image to 640x640. At that size the detector basically cannot see them. SAHI fixes this by slicing the original image into overlapping 640x640 tiles and running YOLO on each tile separately. So that same 10px person becomes around 40px in a tile which is actually detectable. After all tiles are processed, detections get mapped back to full frame coordinates and duplicates at tile edges are removed via NMS.

Parameter	Value	Why
Slice size	640x640	Matches model input , no extra resizing per tile
Overlap ratio	0.2	Persons near tile edges still appear fully in at least one tile
Tiles per frame	~6	Based on image resolution and overlap
Confidence threshold	0.3	Default 0.5 missed too many small persons

Table 4: SAHI settings

3.2. Tracking — ByteTrack

Why ByteTrack and not DeepSORT: DeepSORT needs a ReID model to generate appearance embeddings for each detected person. That model alone is around 100 MB, which would push the pipeline over the size limit. ByteTrack does not need any ReID model. It just uses IoU matching with a two-stage approach. It is faster, lighter, and actually handles occlusions better in my experience with this dataset.

Criterion	ByteTrack	DeepSORT
Needs ReID model?	No	Yes (~100 MB+)
Fits size constraint?	Yes	No
Occlusion handling	Two-stage matching	Single-stage
Speed	Faster	Slower

Table 5: ByteTrack vs DeepSORT

How it works: Each frame, detections are split into high-confidence and low-confidence. High-confidence ones get matched to existing tracks via IoU first. Anything unmatched gets a second chance with the low-confidence detections. This second pass is important for drone footage when a person goes partially behind a tree, their detection confidence drops but they are still there. ByteTrack can still recover that track instead of creating a new ID.

Implementation note: Getting ByteTrack to actually run took multiple attempts (see Section 4.3). Final solution was supervision's `sv.ByteTrack()` which has a clean API and accepts standard numpy arrays directly.

3.3. Camera Motion Compensation (CMC)

The problem: ByteTrack's Kalman filter assumes the camera does not move. But on a drone, the camera is always moving. So even a person standing completely still will shift position in pixel coordinates from one frame to the next, because the drone moved, not the person. The Kalman filter sees this shift and thinks the person moved unexpectedly, which throws off its prediction and eventually causes an ID switch.

Fix — Sparse Optical Flow: Before passing detections to ByteTrack, I estimate how much the camera moved and correct the detection coordinates for it. Steps:

- 3.3.1. Shi-Tomasi corner detection on the previous frame finds 200 stable feature points to track
- 3.3.2. Lucas-Kanade optical flow tracks those points into the current frame
 - 3.3.3. `cv2.estimateAffinePartial2D()` computes an affine transform from the tracked point pairs. This gives the camera's translation and rotation
- 3.3.4. Apply that transform to all detection coordinates before ByteTrack sees them

Why sparse and not dense optical flow: Dense flow computes motion for every pixel accurate but takes around 200ms per frame which is too slow. Sparse flow on 200 corner points takes about 5ms and gives enough information to estimate the global camera motion. **Why LK and not ECC:** ECC is more accurate but slower and

picky about initialisation. For this use case LK is the better tradeoff.

4. Mistakes and What I Learned

4.1 Wrong Validation Sequence

I chose the val sequence before actually looking at the data distribution. After 30 epochs of training the mAP50 came out at 0.00532 which looked terrible. Initially, I suspected an issue with the training process because the validation mAP50 was extremely low. After analysing the dataset, I found that the selected validation sequence contained very few person instances, which made the metric unreliable for evaluating person detection performance.

4.2 Zero Detections on Actual Drone Footage

After training I tested the model on a Shibuya crossing image (near-ground, persons are large) and it detected 24 persons fine. Then I tested it on a real top-down drone image and got zero detections. Persons were around 8-12px tall. This confirmed what I already knew theoretically — full-frame inference does not work at drone altitude and made SAHI non-negotiable.

4.3 ByteTrack Installation — Every Attempt Failed

Atte mpt	What I tried	What happened
1	<code>from boxmot import ByteTrack</code>	ImportError — boxmot changed their API
2	<code>from boxmot import ByteTracker</code>	Same ImportError
3	<code>pip install bytetracker</code>	ModuleNotFoundError even after install
4	<code>ultralytics.trackers.byte_tracker.BYTETracker</code>	Imported but <code>update()</code> wanted ultralytics Results object, not numpy
5 (wo rked)	<code>pip install supervision, sv.ByteTrack()</code>	Clean API, takes standard numpy detections

Table 6: ByteTrack installation attempts

Attempt 4 was the most frustrating — it imported successfully but `tracker.update()` expected an ultralytics internal Results object. Wrapping numpy arrays into that format required going too deep into ultralytics internals, so I abandoned it. `supervision` was the clean solution.

4.4 Colab GPU Ran Out

Training 30 epochs used up the free T4 GPU quota on my main account. I created a new Colab session on a different account, made `best.pt` shareable on Drive, and downloaded it via `gdown`. Small thing but worth noting — running `gdown` without the `--fuzzy` flag downloaded a 68 KB HTML page instead of the actual 6.2 MB model file. The `--fuzzy` flag is needed for Google Drive share links.

4.5 CMC Bug — Functions Defined Inside the Loop

Both CMC functions (`estimate_camera_motion` and `apply_cmc_to_detections`) were inside the main frame loop, so Python redefined them on every frame iteration. Also, `prev_gray = curr_gray` was inside the CMC if-block, so `prev_gray` never updated on frames where CMC did not run. Fixed both by moving function definitions to the top of the file and putting `prev_gray = curr_gray` unconditionally at the end of each frame.

4. Results

Metric	Without CMC	With CMC
Average FPS	3.86	3.41
Min FPS	1.55	1.02
Max FPS	4.79	4.26
Max Track ID at frame 25	158	143
Tiles per frame	~6	~6
Total frames processed	464	464
Model size	6.2 MB	6.2 MB
mAP50 (val)	0.00532*	0.00532*

Table 7: Results on uav0000086_00000_v (464 frames) | Hardware: T4 GPU, Google Colab

Note: * The mAP50 of 0.00532 is not a real reflection of model quality — it is low because the val sequence had almost no persons in it (Section 4.1). The output video on the person-heavy sequence shows the model working correctly.

Per-frame breakdown (T4 GPU): Preprocess ~4ms | SAHI inference ~225ms | Postprocess ~32ms | CMC ~5ms

CMC reduced Max Track ID from 158 to 143 at frame 25 — fewer ID switches. The FPS drop from 3.86 to 3.41 is acceptable for the improvement in tracking consistency. The low minimum FPS (1.02) happens on dense crowd frames where more detections mean more tracker associations.

5. Running on Edge Hardware (Jetson)

The pipeline was designed with this in mind from the start — that is why YOLOv8n was chosen over a larger model, and why ByteTrack was chosen over DeepSORT. To actually deploy on Jetson:

Step 1 — Export to TensorRT:

```
yolo export model=best.pt format=engine device=0
```

TensorRT fuses layers and removes redundant operations. Typically gives 3-5x faster inference than plain PyTorch on the same hardware.

Step 2 — INT8 Quantisation: Cuts memory usage roughly in half with usually less than 2% mAP drop. Need around 500-1000 VisDrone frames as calibration data.

Step 3 — Reduce SAHI tiles on weaker hardware: On Jetson Nano I would increase slice size to 960x960 to get fewer tiles per frame — trades some small-object detection for speed.

Hardware	Expected FPS	Notes
Jetson Orin NX	8-12 FPS	TensorRT INT8 — usable for live surveillance
Jetson Nano	2-4 FPS	Needs frame buffering but workable

Table 8: Estimated Jetson performance

The pipeline suits edge well because YOLOv8n at 6.2 MB fits in Jetson's L2 cache, there is no second neural network (no ReID), and CMC runs on CPU so it does not compete with the GPU during inference.

6. Trade-off Summary

Decision	What I chose	What I did not choose	Why
Model	YOLOv8n	YOLOv8m or larger	Size constraint, needs to run on drone
Tracker	ByteTrack	DeepSORT	No ReID model needed, faster, lighter
Inference	SAHI sliced	Full-frame	Persons are too small at drone altitude
CMC method	Sparse LK flow	Dense flow or ECC	Dense flow is ~200ms/frame, too slow
ByteTrack API	supervision	boxmot or ultralytics internals	Only one that actually worked cleanly

Table 9: Engineering decisions

7. What Does Not Work Well / Future Work

- ID switching still happens during heavy occlusion. ReID would help but that violates the size constraint
- True top-down drone footage (straight down) still causes missed detections. The training data was mostly angled views
- CMC corrects detection coordinates after the fact — ideally it should be integrated into the Kalman filter update step directly, like BoT-SORT does
- 3.41 FPS is not real-time. TensorRT on Jetson Orin NX is the path to getting there
- The mAP numbers are not trustworthy because of the val sequence problem — this needs a proper re-run with correct splits

8. Setup

```
pip install ultralytics==8.4.54 sahi supervision opencv-python Pillow
gdown --fuzzy <best.pt_drive_link>
python preprocess.py
python inference.py --frames /path/to/sequence --output output.mp4
```

Tested on: Python 3.12 | PyTorch 2.10 | CUDA 12.8 | ultralytics 8.4.54 | T4 GPU (Colab)

9. Output

